Ders 12 Genişletilmiş Notu: Dönem Projesi Sunumları 1: Restoran, Lastik Oteli, Sinema, Fitness, Pet Shop, Otel, Araç Kiralama

Ders 12 Genişletilmiş Notu: Dönem Projesi Sunumları 1: Restoran, Lastik Oteli, Sinema, Fitness, Pet Shop, Otel, Araç Kiralama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bu Dersi Nasıl Okumalı?
- 2. Restoran Bilgi Sistemi
- 3. Lastik Oteli Sistemi
- 4. Sinema Otomasyonu
- 5. Fitness Center Otomasyonu
- 6. Pet Shop Bilgi Sistemi
- 7. Otel Otomasyonu
- 8. Araç Kiralama Uygulaması
- 9. Spor Salonu Ağı
- 10. Projeleri Yan Yana Görmek

1. Bu Dersi Nasıl Okumalı?

Bu derste yeni konu anlatılmadı. Dersin tamamı, öğrenci gruplarının dönem boyunca geliştirdiği bilgi sistemi projelerine ayrıldı. Sunumlar büyük ölçüde aynı omurgayı izledi: problem tanımı, fizibilite, gereksinim analizi, sistem analizi, sistem tasarımı, veri tasarımı ve çalışan uygulamanın gösterimi. Bu yüzden ders, analiz ve tasarım araçlarının gerçek bir projede nasıl birleştiğini görmek için iyi bir örnek kümesidir.

Dersin asıl değeri, hocanın sunumların arasına girerek sorduğu sorulardadır. Hoca, ilk gruba yaptığı müdahalede bunların “herkesin sık yaptığı” eksiklikler olduğunu, bu nedenle araya girdiğini söyledi. Aşağıdaki geri bildirimleri de bu gözle okumak gerekir: Bunlar belli bir grubun değerlendirmesi değil, bir bilgi sistemi projesi hazırlayan herkesin kendi çalışmasına uygulayabileceği denetimlerdir.

Hocanın geri bildirimleri üç soruda toplanıyor:

- Analiz yaklaşımı tutarlı mı? Nesne yönelik bir projede yapısal yaklaşımın araçları, yapısal bir projede de nesne yönelik yaklaşımın araçları gereksiz yere çizilmiş mi?
- Modeller birbirini karşılıyor mu? Senaryodaki aktör diyagramdaki aktörle, her senaryo da tasarımdaki bir akışla eşleşiyor mu?
- Çözümleme neye göre yapılmış? Sınıflar problemin kavramlarından mı çıkmış, yoksa ekranlardan mı?

2. Restoran Bilgi Sistemi

2.1 Problem ve Hedef

Bir grup, var olan bir makarna restoranını ele aldı. İşletmede müşteri sayısı artıyor, hizmet alanı ve çalışan kapasitesi ise sınırlı kalıyordu. Mevcut yönetim biçimi zaman ve iş yükü bakımından maliyetliydi. Bu nedenle müşteri ve sipariş takibi için bilgisayar destekli, veri işlemeye dayalı bir sisteme geçilmesine karar verildi.

Yeni sistemin iki hedefi vardı. Birincisi, müşteri sayısı artarken hizmet hızının da artması ve iç organizasyonda sorun çıkmamasıydı. İkincisi, kaydedilen verilerden müşteri profilini ve işletmenin durumunu anlayıp işletmeyi bu bilgilerle geliştirmekti.

2.2 Fizibilite Çalışması

Bu projede fizibilite altı başlıkta incelendi:

- **Teknik:** Sistem Python ile, nesneye dayalı bir yaklaşımla kodlanacak. Arayüz Python kütüphaneleriyle kurulacak. Çalışan sayısı kadar bilgisayar ve sürekli biriken veri için sunucu gerekiyor.
- **Ekonomik:** Yazılım, donanım ve iş gücü giderleri tablo hâlinde tahmin edildi. Toplam gider yaklaşık 35 bin dolar öngörüldü.
- **Zaman:** Yazılım geliştirme modeli olarak waterfall (şelale) seçildi. İş takibi için Google Spreadsheet üzerinde Gantt şeması hazırlandı.
- **Sosyal:** Benzer sipariş takip sistemleri sektördeki başka işletmelerde de kullanılıyor. Sistem müşteri memnuniyetini artıracak ve gelişime yön verecek veri toplayacak.
- **Yasal:** Kişisel veri girişi zorunlu değil. İsim istenebiliyor ama girilmesi şart değil. Girilirse KVKK gereği korunuyor ve üçüncü kişilerle paylaşılmıyor.
- **Yönetim:** Proje, yönetimin isteği ve desteğiyle yürütülüyor. Amaç yönetime kolaylık sağlamak olduğu için sorunlar onların talebiyle gideriliyor.

2.3 Fizibilite Matrisi

Grup, alternatif sistemleri fizibilite matrisiyle karşılaştırdı. Mantık şuydu: Sistemler her fizibilite türü için puanlanır, sistemler arasında farkı yaratan başlık belirlenir ve en yüksek puanı alan sistem seçilir.

Bu projede sosyal ve yönetim fizibilitesi ile bir başka başlıkta sistemler arasında fark çıkmadı. Zaman fizibilitesinde de ciddi bir fark yoktu. Farkı teknik fizibilite yarattı: Ekibin Python bilen kişilerden oluşması birinci sistemi öne çıkardı ve o sistem seçildi.

2.4 Gereksinim Analizi ve Anketler

Gereksinim analizinden iki kritik gereksinim çıktı:

- **Kullanıcı dostu arayüz:** Yoğun saatlerde hızlı hizmet için sistemin kolay kullanılması gerekiyor. Aksi hâlde sipariş hızı ve müşteri memnuniyeti düşer, personel yükü artar. Restoranlarda işçi sirkülasyonu hızlıdır ve çalışanlar farklı demografik yapılardan gelir. Herkesin sistemi hızla öğrenebilmesi bu yüzden önemlidir.
- **Raporlanabilirlik:** Restoran sahibi işletmenin kâra geçtiğini izlemek ister. Ayrıntılı raporlama, hizmet kalitesini, personel performansını ve iş süreçlerini olumlu etkiler.

Bu sonuçlara anketlerle ulaşıldı. Sistemle etkileşen üç gruba, yani müşteriye, çalışana ve yöneticiye ayrı anketler yapıldı. Anketlerde açık ve kapalı uçlu sorular vardı. Çalışan anketinde dikkat çekici bir yöntem kullanıldı: Cevaplar çalışma süresine göre ağırlıklandırıldı. Uzun süredir çalışan birinin verisi daha değerli sayıldı. 6 ay ile 1 yıl arası çalışanların katsayısı 0,8 idi, 1 yıldan uzun çalışanların katsayısı daha yüksekti.

2.5 Kullanım Senaryoları ve Aktör Tutarlılığı

Kullanım durumu diyagramında bir müşteri ve bir yönetici vardı. Müşteri tarafında giriş yapma, masa seçme, sipariş verme ve ödeme yapma, yönetici tarafında restoran verilerine ulaşma yer alıyordu. Senaryolar şu kalıpla yazıldı:

Senaryo	Ön koşul	Ana akış özeti
Giriş yapma	İstenen masa boş olmalı	Çalışan müşteri bilgisini girer, masa durumu kontrol edilir, masa uygunsa giriş tamamlanır, masanın yeni durumu işlenir
Masa seçme	Seçilen masa uygun durumda olmalı	Çalışan müşterinin isteğine göre saat ve masa durumunu değerlendirir, müşteriyi yerleştirir, masa bilgisini girer
Sipariş verme	İlgili masa dolu olmalı	Müşteri siparişine karar verir, çalışan siparişi alıp sisteme girer, bilgi ödeme için tutulur
Ödeme yapma	Masa dolu olmalı ve alınmış sipariş bulunmalı	Çalışan sipariş geçmişine ulaşır, tutarı müşteriyle paylaşır, ödeme alınır, masa boşaltılır

Burada hoca bir tutarsızlığı yakaladı.

Soru: Senaryoların hepsinde aktör çalışan, ama diyagramda müşteri yazıyor. Bu doğru mu?

Cevap: Diyagramda müşteri var, ama arayüzü çalışan kullanacağı için birinci aktör çalışan seçildi.

Grubun gerekçesi, arayüzü kimin kullandığına dayanıyor. Fakat bu kararın diyagrama da yansıması gerekir. Senaryodaki birinci aktör ile diyagramdaki aktör aynı kişiyi göstermelidir.

2.6 Senaryolar Etken ve Özneli Yazılır

Hoca, senaryolar bitince genel bir uyarı yaptı ve bunu yalnız bu gruba değil, herkese söylediğini belirtti. Örnek cümle “ödeme alınır” idi. Ödemeyi kim alacak? Normal şartlarda çalışan alacak. Fakat senaryoları yazmanın amacı, işin ne olduğunu ve kimin ne yaptığını net olarak ortaya koymaktır. Gerçek hayatta da böyle cümleler çok görülür, ama bazen hangi aktörün işle ilgili olduğu anlaşılamadığı için sorun çıkar.

> **Hocanın vurgusu:** Senaryo yazarken mutlaka etken, özneli cümleler yazın. “Ödeme alınır” yerine “Çalışan ödemeyi alır” gibi.

2.7 Nesne Yönelik mi, Yapısal mı?

Grup senaryolardan sonra veri akış diyagramlarına geçince hoca yine durdurdu.

Soru: Nesneye yönelik bir program yazdığınızı söylediniz, kullanım senaryolarıyla başladınız. Niye veri akış diyagramlarıyla devam ettiniz? Sınıfları oluşturmadınız, oysa kodunuzda sınıflar var.

Cevap: Sınıflar yalnız menü kısmında kullanıldı. Programın geneli nesneye dayalı değil.

Hoca bu cevap üzerine “şimdi oldu” dedi. Teknik fizibilitede nesneye dayalı bir yaklaşımdan söz edilmişti, soru-cevapta programın genelinin öyle olmadığı ortaya çıktı. Hocanın sorusu, analizde seçilen araçlar ile kodun gerçek yapısı arasındaki uyumu sınıyordu.

2.8 Örnek Veri Akış Diyagramı

Bu proje, veri akış diyagramının düzeylerini göstermek için iyi bir örnektir. **Taslak düzeyde** tek bir süreç, yani restoran bilgi sistemi vardır. Dış varlıklar müşteri ve yöneticidir. Sisteme müşteri bilgisi, masa bilgisi, sipariş ve ödeme girer. Sistemden müşteriye sipariş tutarı, yöneticiye müşteri verileri çıkar.

Birinci düzeyde tek süreç modüllerine açılır. **İkinci düzeyde** her modülün içi gösterilir:

Modül	Dış girdi	Alt süreçler	Veri deposuna yazılan
Giriş yapma	Müşteri bilgisi	Müşteri ismini alma, müşteri ID'si belirleme	İsim, müşteri verisi olarak
Masa seçme	Seçilen masa bilgisi	Masa bilgisini alma	Masa numarası, masa deposuna
Sipariş verme	Ürün inceleme talebi, sipariş bilgisi	Menüyü görüntüleme, menüden ürün seçme	Seçilen ürün, siparişler deposuna
Ödeme yapma	Ödeme şekli, ücret bilgisi	Tutar hesaplama, ödeme türü seçimi, ücretin alınması	Ödenen tutar, tutar deposuna

Ödeme modülünde veri akışı zincir hâlindedir: Sipariş bilgisi depodan okunur, tutar hesaplamada kullanılır ve "ödenecek tutar" olarak çıkar. Sonra ödeme türü seçilir, ücret alınır ve "ödenen tutar" kaydedilir.

2.9 Varlık-İlişki ve Yapı Diyagramları

Varlık-ilişki diyagramında müşteri, restoran, çalışan, bilgi işlem ve yönetim bileşenleri vardı. Müşterinin masa numarası, isim ve seçilen ürün gibi bilgileri, restoranın masaları, ürünleri ve ödeme yöntemleri bulunuyordu. Çalışan siparişi ve tutarı bilgi işleme raporluyor, bilgi işlem de giriş-çıkış saatlerini işleyip yönetime raporluyordu.

Yapı diyagramı sistemin modül hiyerarşisini ve sırasını şöyle veriyordu:

- Restoran hizmet sistemi
  - Müşteri girişi: müşteri ismi ve masa numarası gerekir, onaylanırsa devam edilir
  - Menü görüntüleme ve sipariş verme: sipariş onaylanırsa devam edilir
  - Sipariş tutarını hesaplama
  - Ödeme yöntemi: nakit veya kart
  - Ödenen tutarı görüntüleme

2.10 Görev Dağılımı

Uygulama açılırken hoca nasıl bir organizasyon yapıldığını, görev dağılımının nasıl olduğunu sordu. Grup bunun raporda bulunduğunu söyledi. Hoca, ekibi ve görev dağılımını tanıtmanın sunumun da bir parçası olması gerektiğini belirtti.

2.11 Uygulamanın Gösterimi

Demoda çalışan, müşterinin dört numaralı masayı istediğini girdi. Menüde makarna, içecek ve tatlı seçenekleri vardı. Ürünler eklenip silinebiliyor, işlenen veriler terminale de yansıyordu. "İleri" denildiğinde sistem siparişi hemen almıyordu, çünkü çalışanın bilgiyi müşteriden teyit edip onay kutusunu işaretlemesi gerekiyordu. Sonra kredi kartı ya da nakit seçilerek ödeme yapılıyordu.

Veri tabanında her tarih için ayrı bir tablo tutuluyordu. Satırlarda müşteri ID'si, isim, masa numarası, giriş-çıkış saati, tarih, sipariş ve toplam tutar vardı. Yönetim kodu, girilen ay ve yıl için ayın birinci gününden son gününe kadar verileri çekip iki grafik çiziyordu: günlere göre müşteri sayısı ve günlere göre kazanç. Böylece yönetim hangi gün daha yoğun olduğunu görebiliyordu.

Soru: Aynı anda birkaç masa sipariş veremiyor mu?

Cevap: Verebiliyor. Her çalışanda bu sistemden bir tane olacağı için her masa için ayrı bir sistem varmış gibi oluyor. 8 çalışan ve 8 masa var, yani her masaya ayrı bir garson bakıyor.

Hoca bu varsayımı “istihdam yaratmışsınız” diyerek esprili biçimde karşıladı.

> **Ek bilgi (derste anlatılmadı):** Bu varsayımda eşzamanlılık yazılımla değil, masa başına bir çalışan ve bir kurulumla karşılanmış oluyor. Masa başına ayrı çalışan ve bilgisayar, eşzamanlı sipariş sorununu büyük olasılıkla çözmez, yalnız donanım ve personel maliyetini artırır. Çok kullanıcılı bir sistemde genellikle bütün istemciler ortak bir veri tabanına bağlanır ve eşzamanlılık kontrolü veri tabanı düzeyinde yapılır.

3. Lastik Oteli Sistemi

3.1 Problem

Bir grup, lastik saklama hizmeti veren bir işletmeyi ele aldı. Lastik oteli, lastikleri belli bir süre müşteriden alıp saklayan ve sonra geri veren bir hizmettir. Lastik sayısı artınca işletme bu ihtiyacı karşılayamaz oldu. Önerilen çözüm, giren çıkan müşteri lastiklerinin ve satılabilecek lastiklerin bilgilerini tutmak, stok takibi yapmak ve günlük, haftalık, aylık raporlar üretmekti. Bunlar elektronik bir kayıt sistemi ve veri tabanıyla yapılacaktı.

Ekip yapısında müşteri, proje yöneticisi, sistem analisti ve yazılım geliştiriciler vardı. Beş toplantı yapıldı: üçü müşteriyle istekleri öğrenmek, ikisi ekip içinde geliştirme ve tasarım için.

3.2 Fizibilite Hesabı

Teknik fizibilitede nesneye yönelik programlama hedeflendi. Ekibin deneyimi nedeniyle Java, veri tabanı olarak MySQL ve Windows seçildi. İşletmede daha önce bilgisayar ya da bilgi sistemi bulunmadığından gerekli asgari donanım raporda belirtildi.

Ekonomik fizibilite, bu dersteki en net hesaplardan biridir. Özel yazılım ya da donanım gideri yoktu, gider olarak yalnız ekip üyelerinin ücreti hesaba katıldı.

- **Verilen:** Saatlik ücret 150 TL. Gantt şemasındaki toplam çalışma 520 saat. Satış fiyatı 120 bin TL olarak belirlendi.
- **Gider:** $520 \times 150 = 78\,000$ TL
- **Fark (ara adım eklendi):** $120\,000 - 78\,000 = 42\,000$ TL

Grup, bu nedenle projeyi maliyet etkin buldu. Zaman fizibilitesinde kullanılan araç Gantt şemasında süreleri günler yerine saat olarak gösteriyordu, 520 saatlik toplam da buradan alındı.

Operasyonel fizibilitede hedef kitle lastik oteli ve lastik satışı yapan iş yeriydi. Kullanım kolaylığı, zaman kaybının azalması ve verimlilik önemliydi. Muhasebe, ürün takibi ve stok işlemleri kolaylaşacağı için yönetim desteği de bekleniyordu. Yasal fizibilitede özel ticari lisans gerekmediği belirtildi. TC kimlik numarası ve isim gibi kişisel bilgiler tutulduğu için 6698 sayılı Kişisel Verilerin Korunması Kanunu’na uyulduğu söylendi.

Fizibilite matrisinde üç öneri teknik, ekonomik, zaman ve operasyonel açıdan ağırlıklı olarak puanlandı. Bu projede ekonomik ve zaman ağırlıkları daha yüksekti, teknik fizibilite de önemliydi. İkinci öneri 89 puanla seçildi.

Gereksinim analizinde ön inceleme ve fizibilite bulguları bir araya getirildi, ayrıca bir anket yapıldı.

3.3 Senaryolar

Aktörler müşteri ve yöneticiydi. Müşterinin lastik satın alma, depoya lastik emanet etme ve depodan lastiği geri alma işleri vardı. Yönetici şifreyle giriş yapıyor, depodaki lastikleri listeliyor, lastik ekliyor, satıyor, emanet kabul ediyor ve kontrollerini yapıyordu. Toplam 10 senaryo vardı.

Hoca, sunumda her senaryonun tek tek okunmaması gerektiğini, senaryonun ne yaptığının ve önemli noktaların söylenmesinin yeterli olduğunu belirtti. Örnek olarak lastik satın alma senaryosu anlatıldı:

- **Birinci aktör:** Müşteri
- **Beklenti:** Lastiği sorunsuz satın almak

- **Ön koşul:** Müşteri sisteme kayıt olmuş ve giriş yapmış olmalı
- **Son koşul:** Müşterinin talep ettiği lastik depoda bulunmalı
- **Ana akış:** Müşteri istediği lastiğin özelliklerini girer, depodakilerle karşılaştırılan lastik müşteriye tahsis edilir
- **Alternatif akış:** Lastik depoda yoksa sistem hata verir ve ana menüye dönülür

  Ders sonrası not: Sunumda "lastiğin depoda bulunması" son koşul olarak söylendi. Bu durum senaryo başlamadan doğrulandığı ve alternatif akışta ele alındığı için büyük olasılıkla son koşul değildir. Son koşul, akış bitince sağlanmış olması gereken durumu, yani lastiğin müşteriye tahsis edilmiş olmasını anlatmalıdır.

3.4 Sınıf Diyagramları

Grup kod nesne tabanlı olduğu için önce kavramsal, sonra ayrıntılı UML sınıf diyagramı çizdi. Yapı şöyleydi:

Sınıf	Görev
`Person` (soyut)	`Customer` ve `Manager` sınıflarının üst sınıfı
`Manager`	Fazladan şifre tutar
`Tire`	Marka, model, sahibin adı ve TC'si, fiyat ve her lastiğe özgü ID tutar
`Warehouse`	Lastikleri liste hâlinde bulundurur
`Operations`	Depoyu kullanır, veri tabanı işlemlerini yapar; lastik ekleme, satma, emanet alma ve geri verme burada toplanır
`Report`	Yöneticiye yapılan satışları ve işlemleri ücret, kişi ve zaman bilgisiyle listeler

3.5 Örnek Ardışıl Diyagram

Ardışıl diyagramlarda senaryoların gerçeklenmesi gösterildi. Müşterinin lastik satın alması şu akışla anlatıldı:

```
Müşteri  -> Customer   : ad ve TC ile nesne oluşturulur
Customer               : listele lastikler metodu, for döngüsüyle
Müşteri  -> Sistem     : seçilen lastiğin ID'si girilir
Sistem                 : lastik satılabilir mi, emanet mi kontrol edilir
  [emanet ise]  Müşteri <- "this tire cannot be sold"
  [satılabilir ise]
    Sistem -> Database : veri tabanı işlemleri
    Sistem -> Warehouse: find tire by id, lastik nesnesi döner
    Sistem -> Warehouse: remove tire, listeden çıkarılır
    Müşteri <- "tire successfully sold"
```

Dalın nedeni problemin kendisinden gelir. Depoda hem şirketin satabildiği lastikler hem de müşterilerin emanet bıraktığı lastikler karışık durur.

3.6 Senaryo Sayısı ve Ardışıl Diyagram Sayısı

Bu noktada hoca dersin en önemli geri bildirimlerinden birini verdi. Analiz modelinde 10 senaryo vardı, ama ardışıl diyagramlarla gösterilen tasarımda yalnız 6 sorumluluk görülüyordu. Hoca bunu şöyle yorumladı: Senaryolar biraz fazla parçalanmış olabilir, ama tasarımda her senaryoyu karşılayan bir şey birebir bulunmuyor. Senaryoları gerçeği daha iyi yansıtacak şekilde bölmek gerekir.

Kural şöyle kuruluyor. Her senaryo bir akış, yani bir sorumluluk demektir. Dolayısıyla en az bir ardışıl diyagram üretir. Hatta alternatif akışların ayrı ayrı gösterilmesi gerektiğinde tek diyagram yetmeyebilir.

$$\text{ardışıl diyagram sayısı} \geq \text{kullanım senaryosu sayısı}$$

Bu projede sayılar $6 < 10$ olduğu için eşitsizlik sağlanmıyor ve 4 senaryonun karşılığı açıkta kalıyor.

> **Hocanın vurgusu:** Analizdeki her senaryo, tasarımda en az bir ardışıl diyagram üretmelidir. Sorumluluk sayısı en az senaryo sayısı kadar, hatta daha fazla olmalıdır.

3.7 Veri Tabanı Tasarımı

Hoca, uygulamaya geçilirken veri tabanı tasarımına dair bir içerik görmediğini, yalnız "MySQL kullandık" denildiğini belirtti. Hocanın sözü, veri tabanı ürününü adlandırmanın veri tasarımının yerini tutmadığına işaret ediyor.

3.8 Uygulamanın Gösterimi

Uygulamada yönetici ve müşteri için iki ayrı giriş vardı. Yönetici ID ve parolayla giriyor, lastik ekleyip silebiliyor, aylık kazancı hesaplayabiliyordu. Eklenen lastik MySQL'de tarihiyle görünüyordu. Aylık kazanç, raporlar tablosundaki kayıtların tarih alanından o aya ait fiyatlar toplanarak bulunuyordu. Demoda 5. ayın kazancı 6450 TL çıktı.

Soru: Lastik bırakırken de alırken de para alınıyor mu, her işlemde fatura mı kesiliyor?

Cevap: Lastiği bırakırken para alınıyor, geri alırken alınmıyor. Geri alma işlemi rapora sıfır fiyatla yansıtılıyor.

Satın almada önemli olan, lastiğin şirkete mi müşteriye mi ait olduğuydu. Müşteriye aitse işlem yapılmıyor, şirkete aitse satış gerçekleşip rapora yansıyordu. Emanet bırakmada lastiğin modeli, markası ve kaç gün kalacağı alınıyor, ücret gün sayısına göre hesaplanıyordu. Geri almada müşterinin TC'siyle eşleşen lastik varsa iade yapılıyordu.

Soru: Müşterilerin adı aynı olursa ne olacak?

Cevap: Eşleştirme addan değil TC'den yapılıyor, bu yüzden sorun çıkmaz.

Müşteri işlemlerinin sonunda şikâyet dilekçesi yazmak için bir form açılıyordu, fakat yazılanlar bir yere aktarılmıyordu. Hoca bunu esprili biçimde "en azından gazını alıyorsunuz" diye yorumladı.

4. Sinema Otomasyonu

4.1 Problem ve Süreç

Bir grup, bir sinema işletmesinin bütün fiziksel işlemlerini dijitale taşıma talebi üzerine bir sistem tasarladı. Sistemde kullanıcı işlemleri, film bilgisi görüntüleme, bilet satın alma ve iptal, film yorumları ve sinema sahibine dilek, şikâyet, öneri iletme paneli vardı.

Beş toplantı yapıldı. Birincisinde müşterinin beklentileri alındı ve müşterinin kendi müşterilerine iletmesi için bir form verildi. İkincisinde formdan gelen beklentiler sistem gereksinimleriyle birleştirildi. Üçüncüsünde Gantt şeması, zaman planı, diyagramlar ve yazılım süreci başladı. Dördüncüsünde ürün son hâlini aldı, beşincisinde müşteriye sunuldu.

4.2 Fizibilite

Fizibilite matrisinde üç framework karşılaştırıldı. Puanlar sonuçlanınca hem ekip hem sistem gereksinimleri için en uygun seçeneğin Django olduğu görüldü. Gantt şemasında beş aşama beş renkle gösterildi: ön hazırlık, fizibilite, sistem analizi ve tasarımı, kodlama ve final.

- **Teknik:** Python, Django ve Django ile hazır gelen SQLite. Geliştirme Windows üzerinde VS Code ile yapıldı.
- **Sosyal:** Kullanıcı kolaylığına ve arayüzün iyi olmasına odaklanıldı.
- **Ekonomik:** Giderler en az düzeyde tutuldu, tahmini bütçe 4000 dolardı.
- **Yönetim:** Müşteri, kendi müşterilerinin memnuniyetini ve bilet satış durumunu izleyebilecekti.
- **Yasal:** Kişisel bilgi alındığı için 6698 sayılı Kanun'a dikkat edildi.

Ekonomik fizibilite anlatılırken grup kendini “yeni bir girişim” diye tanımladı. Hoca burada kavramı düzeltti: Bir girişim tutarsa artık normal şirket olur, ona girişim denmez. Bütün girişimler zaten yenidir, yani “yeni girişim” gereksiz bir ifadedir.

4.3 Veri Akış Diyagramı

Taslak düzeyde kullanıcı ve sinema sahibi firma, sinema sistemiyle ilişki kuruyordu. Birinci düzeyde kullanıcının kayıt olma, film görüntüleme, bilet alma, yorum yapma, dilek-şikâyet-öneri bildirme ve bilet işlemleri, firmanın ise sinema ekleme işi görüldü. Hoca, ikinci düzeyde her şeyi tek tek okumak yerine özellikle bölünmesi gereken yerlerin anlatılmasını istedi.

Bu nedenle bilet alma süreci örneklendi. İkinci düzeyde süreç şu adımlara ayrılıyordu:

1. Kullanıcı film seçer, tarih seçer, seans seçer.
2. Bilet türünü ve sayısını seçer.
3. Oturacağı koltukları seçer.
4. Bilet ücreti firmadan çekilir ve ödenecek tutar hesaplanır.
5. Kullanıcıdan kart bilgisi alınır ve ödeme yapılır.
6. Bilet ayrıntıları biletler deposuna kaydedilir.

Bilet işlemlerinde kullanıcı aldığı biletleri görüntülüyor, tarihi geçmediyse iade edebiliyordu. Firma sinema eklerken salon, film ve seans ekliyordu.

4.4 Hangi Diyagramlar Gerekli?

Sıra kullanım durumu diyagramına gelince hoca, bilgilerin veri akış diyagramında zaten anlatıldığını söyledi. Grup eksik kalmasın diye bu diyagramı da eklediğini belirtti.

> **Hocanın vurgusu:** Python ile yazıp nesne yönelik yapmadıysanız kullanım durumu diyagramı ve kavramsal sınıf diyagramı çizmeniz gerekmez.
>
> **Ek bilgi (derste anlatılmadı):** Django’da modeller Python sınıfları olarak yazılır. Bu yüzden kodda sınıf bulunması, projede nesne yönelik çözümleme yapıldığını tek başına göstermez. Hocanın bu projeyi nesne yönelik saymamasının nedeni büyük olasılıkla budur: Belirleyici olan, kavramların ve sorumlulukların nesne yönelik olarak modellenmiş olmasıdır.

4.5 Örnek Varlık-İlişki Diyagramı

Diyagramda dikdörtgenler varlıkları, çevresindeki yuvarlaklar öz nitelikleri gösteriyordu. Altı çizili öz nitelikler her varlık için tanımlayıcıydı. İlişkiler şöyleydi:

Varlık	İlişki	Varlık
Müşteri	satın alır	Bilet
Film	belirli seans ve salonlarda gösterilir	Seans, Salon
Bilet	rezervasyon	Seans
Salon	sahiplik	Koltuk
Sinema	sahiptir	Salon

4.6 Yapı Diyagramı

Yapı diyagramında kullanıcı kayıt oluyor, giriş yapıyor ya da üye olmadan devam edebiliyordu. Yönetici girişi film, sinema, salon ve seans eklemeye, normal kullanıcı girişi kullanıcı paneline gidiyordu. Panelden film görüntüleme, dilek-şikâyet oluşturma, yorum yapma, bilet seçme ve ödeme işlerine geçiliyordu. Ödeme sonrası otomatik e-posta gidiyor, bilet iadesi de yapılabiliyordu.

> **Ders sonrası not:** Sunumda “UML’nin daha genel bir tanım olduğu, yapı diyagramının da onun içinde olduğu” söylendi. Bu doğru değildir. Yapı diyagramı yapısal tasarımın aracıdır, UML diyagramlarından biri değildir. Hocanın nesne yönelik projelerde yapı diyagramını gereksiz bulması da bu ayrımdan gelir.

4.7 Uygulamanın Gösterimi

Uygulamada kullanıcı ad, soyad, kullanıcı adı, e-posta ve şifre girip KVKK onayı vererek üye oluyordu. Ana sayfada vizyondaki filmler görünüyor, film sayfasında vizyon tarihi, yönetmen, puan ve türler yer alıyordu. Giriş yapılınca fragman ve yorum seçenekleri açılıyordu.

Bilet almada önce mekân, tarih, saat ve altyazı ya da dublaj bilgisiyle seans seçiliyordu. Sonra bilet sayısı giriliyordu, örnekte bir tam ve bir öğrenci bileti alındı. Dolu koltuklar seçilemiyor, seçilen koltuk sayısı bilet sayısına eşit olmak zorundaydı. Uygula düğmesi ancak o zaman etkinleşiyordu.

Bu projenin en öğretici tasarım kararı süre sınırıydı. Bilet alma işlemi 5 dakika içinde tamamlanmak zorundaydı, çünkü salon durumu sürekli güncellenebilir ve aynı koltuğu birden fazla kişinin alması önlenmelidir. Ödemede yanlış bilgi girilirse uyarı veriliyordu. Ödeme sonrası her bilet için ayrı e-posta geliyordu. Biletlerim bölümünden iade yapılıyor, iade öncesinde onay isteniyor, sonrasında yine e-posta gidiyordu. Django'nun hazır yönetici sayfasından film, sinema, salon, seans ve biletler yönetiliyordu.

5. Fitness Center Otomasyonu

5.1 Problem ve Organizasyon

Bir grup, spor salonu için bir yönetim sistemi geliştirdi. İşletme sahibi, antrenör ve sekreter rollerini, bu rollerin üye işlemlerini ve finansal işlemlerin raporlanmasını içeren bir sistem isteniyordu.

Ekipte proje yöneticisi, sistem analistleri, frontend ve backend geliştiricileri vardı. Altı toplantının içeriği sürecin iyi bir özetidir:

1. Müşteriden sistem için gerekli bilgiler alındı, rol dağılımı yapıldı.
2. Modüller ve teslim tarihi belirlendi.
3. Yazılım dili belirlendi, analistler iş paylaşımı yaptı.
4. Müşteriye UML kavramsal diyagram sunuldu, tasarım tartışıldı.
5. Tasarım müşteriye sunuldu. Müşteri daha açık renk istedi, değiştirildi ve gerçekleştirmeye geçildi.
6. Gerçekleştirme bitti, sistem müşteriye teslim edildi ve olumlu dönüş alındı.

5.2 Örnek Fizibilite Matrisi

Bu sunum, fizibilite matrisinin neden işe yaradığını adım adım gösterdi. Üç sistem önerildi: Flutter ve Firebase, Java ve MySQL, C# ve Microsoft SQL Server. Karşılaştırmanın mantığı şöyleydi:

Başlık	Sistem 1: Flutter, Firebase	Sistem 2: Java, MySQL	Sistem 3: C#, MS SQL Server
Teknik	Ekip için eğitim gerekir	En yüksek puan	Ekip için eğitim gerekir
Zaman	Eğitim süresi eklenir	En yüksek puan	Eğitim süresi eklenir
Sosyal	En yüksek puan: aynı kod hem masaüstü hem Android	Platform bağımsız olduğu için üçüncü sistemin önünde	Yalnız masaüstü
Ekonomik	Eğitim gideri, Firebase aboneliği pahalı	En yüksek puan	Eğitim gideri, lisans pahalı

Sonuçta ikinci sistem toplam 66 puanla seçildi. Bir seçeneğin bir başlıkta en yüksek puanı almasının yetmediği burada görülüyor: Flutter sosyal fizibilitede önde olmasına rağmen ekibin eğitim ihtiyacı teknik, zaman ve ekonomik başlıkların üçünde de onu geride bıraktı.

5.3 Seçilen Sistemin Fizibilitesi

- **Teknik:** Windows 11, Java, NetBeans ve MySQL. Raporlar için Microsoft 365, toplantılar için Zoom, diyagramlar için Draw.io, Gantt için ayrı bir araç kullanıldı.
- **Zaman:** Analiz 14, tasarım 12, gerçekleştirme 27 gün: $14 + 12 + 27 = 53$ gün.
- **Sosyal:** Hedef kullanıcılar salon çalışanlarıydı. Temel bilgisayar bilgilerinin olduğu öğrenilince masaüstü uygulamasına karar verildi. Görüşmelerde basit bir arayüz istendi.
- **Yönetim:** İş yükünü hafifletiyor, raporlamayı ve analizi kolaylaştırıyor. Dijital ortama geçiş kâğıt ihtiyacını azaltıyor, veri güvenliğini kolaylaştırıyor.
- **Yasal:** Veri işleme ve koruma için Kişisel Verilerin Korunması Kanunu, ticari sırlar ve çalışan yükümlülükleri için Bilgi Edinme Kanunu ile Türk Ceza Kanunu, ayrıca yazılım lisansları ve görsellerin telif hakları incelendi.

Ekonomik fizibilite: Geliştirme, kullanma ve çalışma giderleri toplanarak yatırım miktarı bulundu, sonra geri ödeme süresi yöntemi uygulandı. Gelirin gideri karşıladığı nokta arandı.

- **Verilen:** Aylık abonelik 1000 TL, ortalama 50 müşteri, geri ödeme 3 ay.
- **Aylık gelir (ara adım eklendi):** $1000 \times 50 = 50\,000$ TL
- **Üç aylık gelir (ara adım eklendi):** $3 \times 50\,000 = 150\,000$ TL

5.4 Ekranlara Göre Sınıf Olmaz

Kullanım durumu diyagramında üç aktör vardı. Sekreter müşteri ve rezervasyon işlemlerini yapıyor, salonda satılan ürünleri giriyordu. Antrenör ders ekliyor ve spor programı oluşturuyor, programı PDF'e dönüştürebiliyordu. İkisi de arıza ve temizlik gibi konularda yöneticiye rapor iletebiliyordu. Yönetici raporları izliyor, personel işe alıp çıkarıyordu.

Sıra kavramsal sınıf diyagramına geldiğinde sınıflar "yönetici main UI", "yönetici personel UI", "yönetici rapor UI", "yönetici işe al UI" gibi adlandırılmıştı. "Yönetici işe al UI" sınıfının özellikleri de isim, numara, TC kimlik numarası ve telefon numarasıydı. Hoca burada durdu ve bunun aslında işlevsel çözümleme olduğunu söyledi.

Nesne yönelik çözümlemede modellenen şey yönetici, salon, seans, ders gibi kavramlardır. Arayüz ayrı bir şeydir ve hepsi aynı bileşenleri kullanır. Sınıfları "şu işlemler", "şu ekranlar" diye bölmek iyi bir tasarım vermez.

> **Hocanın vurgusu:** Nesne yönelik bir sistemin analizini ve tasarımını yaparken kavram çözümlemesi yapın, işlev çözümlemesi yapmayın. Sınıflarınızı işlemlere ve ekranlara göre bölmeyin.

Grup, sınıfları aslında ayırdığını, fakat işlemleri doğrudan veri tabanı üzerinden yaptıkları için sınıf nesnelerini pek kullanmadığını, bütün kodun arayüze yazıldığını belirtti. Hoca buna katmanlar üzerinden cevap verdi.

> **Hocanın vurgusu:** İş katmanıyla sunum katmanını ayırmanız gerekir. Ön yüzde iş kodu çalışmamalıdır.

5.5 Tasarım Sınıf ve Ardışıl Diyagramları

Tasarım sınıf diyagramında giriş ekranının find from database işlevi, çalışanın kategorisini bulup doğru ekrana geçiriyordu. Yöneticide display report ile rapor inceleme ve işe alıp çıkarma, sekreterde rapor oluşturma ve sell supply ile satış, antrenörde add course ve trainer workout plan işlevleri vardı.

Giriş için ardışıl diyagram şöyle işliyordu: valid login değişkeniyle kişinin gerçek bir salon personeli olup olmadığına bakılıyor, öyleyse pozisyonuna göre doğru ekrana yönlendiriliyordu. Ardından varlık-ilişki diyagramı sunuldu.

5.6 Uygulamanın Gösterimi

Sekreter olarak girişte abonelik işlemleri, rezervasyon işlemleri, antrenör saatleri ile müşteriler arasında eşleştirme, satış girişi ve öneri-şikâyet ekranları vardı. Satış girişi yöneticiye gösterilecek rapora düşüyordu. Antrenör girişinde ders ekleme ve ders programı oluşturma ile bir beden kitle indeksi hesaplama arayüzü bulunuyordu. Hazırlanan spor programı PDF

çıktısı veriyordu. Yönetici girişinde personel kayıtları, işe alma, TC kimlik numarasıyla işten çıkarma ve veri tabanındaki bütün raporları listeleyen rapor inceleme bölümü vardı.

6. Pet Shop Bilgi Sistemi

6.1 Problem ve Kapsam

Bir grup, evcil hayvan mağazalarının ürün yönetimi ve satışta yaşadığı zorlukları ele aldı. Stok işlemlerinin elle yürütülmesi büyük zaman kaybı yaratıyordu. Sistemde satış elemanı sade bir arayüzle giriş yapıyor, ürün yönetimi paneliyle stoğu izliyordu. Müşteri kaydı oluşturulduğu için satışta bilgiler tekrar alınmıyor, TC bilgisiyle satış yapılıp fatura oluşturuluyordu. Fatura PDF olarak kaydediliyor, istenirse yazıcıdan çıktı alınıyordu. Müşterinin özel isteği üzerine hayvan satışı ve sahiplendirme için bir hayvan paneli de eklendi.

6.2 Fizibilite

Fizibilite matrisinde ekibin yetenekleri göz önünde tutularak üç sistem planlandı: Linux ve C#, Windows ve Java, macOS ve Java. Ekip Java bildiği için eğitim gerekmiyordu. Birinci sistemde eğitim gideri, üçüncüde işletim sistemine uygun ürün alımı ekstra gider yaratıyordu. Mağaza yöneticisi ve çalışanlar da Windows'a daha aşinaydı. Bu nedenlerle ikinci sistem seçildi. Java ve MySQL kullanıldı.

- **Sosyal:** Hedef kitle mağaza yöneticisi ve çalışanlardı, onların taleplerine göre ilerlendi.
- **Yönetim:** Yönetim, fizibilite raporlarını ve ön inceleme sonuçlarını görerek projeyi destekledi.
- **Yasal:** Hayvanları Koruma Kanunu, Kişisel Verileri Koruma Kanunu ve lisans sözleşmeleri dikkate alındı.
- **Zaman:** Gantt şemasına göre analiz, tasarım, geliştirme ve test süreçlerinden sonra yaklaşık iki ay sonunda teslim yapıldı.
- **Ekonomik:** Gider yalnız gerekli lisans ve hizmetlerden oluşuyordu. Değer 123 400 idi ve maliyetin 2,5 ayda geri kazanılması bekleniyordu.

Geri kazanım için gereken aylık tutar: $123\,400 / 2{,}5 = 49\,360$ (ara adım eklendi).

6.3 Senaryolar

Sistem teslim edildiğinde yalnız yönetici giriş yapabiliyordu. Yönetici istediği kadar satış elemanı ekliyor, elemanlar da tüm yetkiye sahip oluyordu. Kullanım durumları eleman, müşteri, hayvan, ürün ve faturalandırma başlıklarına ayrılıyordu. Örnek senaryolar şunlardı:

- **Hayvan sahiplendirme**
  - Birinci aktör: satış elemanı
  - Ön koşul: Eleman sisteme doğru giriş yapmış olmalı, hayvan sistemde bulunmalı
  - Son koşul: Hayvan başarıyla sahiplendirilmiş olmalı
  - Akış: Hayvanlar düğmesine tıklanır, hayvan seçilir, sahiplendir düğmesine basılır. Hayvanın ID'si eksiyle çarpılır, sahiplendirilen hayvanlar listenin en üstünde görünür.
- **Başarılı giriş**
  - Birinci aktör: satış elemanı
  - Akış: Kullanıcı adı ve şifre doğru girilir, hoş geldin mesajı çıkar
- **Hatalı giriş**
  - Alternatif akış: Bilgiler yanlış girilir, "hatalı kullanıcı adı veya şifre" mesajı gelir, hata ekranı kapatılınca yeniden denenir

6.4 Sınıf, Ardışıl ve Varlık-İlişki Diyagramları

Kavramsal sınıf diyagramı giriş ekranından başlıyor, doğru giriş sonrası hayvan ekranı, oradan kullanıcı, müşteri, fatura ve ürün ekranlarına geçiliyordu. Tasarım tarafında her panelin ekleme, silme, düzenleme işlevleri vardı. Hayvan panelinde sahiplendirme, fatura panelinde müşteri TC'si ve ürün ID'sine göre fatura oluşturma, stoktan düşme ve PDF kaydetme işlevleri bulunuyordu.

Faturanın ardışıl diyagramı şöyleydi:

```
Eleman -> Fatura paneli : ürün ID'si ve miktar
Panel  -> Database      : ürün stok miktarı sorgulanır
  [miktar > stok]  Eleman <- hata mesajı
  [miktar <= stok] ürün faturaya eklenir
Eleman -> Panel         : fatura kaydet, ürün stoktan düşülür
Eleman -> Panel         : fiş yazdır, fatura PDF olarak kaydedilir
```

Varlık-ilişki diyagramında beş bileşen vardı. Birincil anahtarların altı çizildi:

Varlık	Birincil anahtar
Ürün	ID
Kullanıcı	Kullanıcı adı
Hayvan	ID
Müşteri	TC kimlik numarası
Fatura	Kendine özgü birincil anahtarı yok

> **Ek bilgi (derste anlatılmadı):** Hoca aşağıdaki veri kararlarına ayrıca yorum yapmadı, ama bunlar büyük olasılıkla iyileştirilebilir. Sahiplendirmeyi ID'nin işaretiyle kodlamak yerine ayrı bir durum alanı tutmak, anahtarın tek görev üstlenmesini sağlar. Fatura gibi sonradan başvurulacak bir kayıt için ayrı bir birincil anahtar faydalıdır. Restoran projesindeki gibi her gün için ayrı tablo açmak yerine tarih alanlı tek tablo, aylık sorguları kolaylaştırır.

6.5 Uygulamanın Gösterimi ve Hata Mesajı

Demoda sahiplendirme yapılınca ID'nin başına eksi geliyordu. Yeni kullanıcı eklenip giriş yapabildiği gösterildi. Müşteri silindi, ürün fiyatı 300'den 400'e güncellendi ve yeni ürün eklendi. Faturada müşteri TC'si, ürün ID'si ve miktar girildi, kaydedilince stok düştü: Stokları 50 olan iki üründen 10'ar tane satılınca $50 - 10 = 40$ kaldı.

Hoca burada sınır durumunu sınadı ve 50 tane ürün satılmasını istedi. Sistem "stoktan fazla girildi" hatası verdi. Hoca kontrolün çalıştığını kabul etti, fakat mesajın daha iyi olabileceğini gösterdi: Sistem stoğu zaten sorguladığına göre kullanıcıya "stokta 40 tane var" diyebilir.

> **Hocanın vurgusu:** Stoğu zaten sorguluyorsunuz, kullanıcıya şunu da söyleyin: Stokta 40 tane var.

PDF fatura gösterilirken hoca, faturada resmî kurumu çağrıştıran bir öğenin kaldırılmasını esprili bir dille önerdi.

7. Otel Otomasyonu

7.1 Kapsam ve Fizibilite

Bir grup, beş modüllü bir otel otomasyonu geliştirdi: kayıt işlemleri, yemek takvimi, etkinlik takvimi, istatistiksel veriler ve oda bakım durumu. Temel amaç, işlevleri tek merkezde birleştirip otel yönetimini kolaylaştırmaktı. Müşteri bulunamadığı için ekip içinden biri müşteri seçilerek süreç simüle edildi ve dört toplantı raporu çıktı: proje tanıtımı, gereksinimlerin belirlenmesi, tasarım sunumu ile onayı, ilerleme değerlendirmesi ve aksiyon planı.

Maliyet fizibilitesinde her ekip üyesine ücret biçildi, diğer giderler de eklenerek iki aylık proje maliyeti hesaplandı. Teknik fizibilitede, birçok cihazda çalışması için Java ve onunla uyumlu olduğu için MySQL seçildi. Donanım önerileri ortalama değerlerle de karşılanabiliyordu, ağ bağlantısı ise veri tabanına erişim için zorunluydu.

Yasal fizibilite iyi bir bakış açısı sundu. Sistemi satın alan otel, kendi müşterilerine karşı yükümlülük taşır: müşteri bilgilerinin işlenmesi, veri güvenliği, aydınlatma yükümlülüğü, müşteri hakları ve veri ihlalleri. Bir müşteri verilerinin ne kadar süre, hangi koşulda saklandığını ve üçüncü kişilerle paylaşılıp paylaşılmadığını sorabilir, otel de kanun çerçevesinde cevap vermek zorundadır. Grup, projeyi buna uygun geliştirdiğini söyledi.

Gantt şemasında planlanan süre mavi, gerçekleşen süre turuncu gösterildi.

7.2 Senaryo Örneği

Toplam 11 senaryo vardı. Sistemi yalnız çalışan kullandığı için birinci aktör çalışandı. Örnek senaryo oda kaydı silmeydi:

- **Birinci aktör:** Çalışan
- **Beklenti:** Odayı sistemden sorunsuz silmek
- **Ön koşul:** Sistem açık ve çevrim içi, seçilen oda dolu
- **Son koşul:** Oda kaydı başarıyla silinmiş
- **Ana akış:** Çalışan dolu bir oda seçer, kaydı siler, işlem başarıyla gerçekleşir
- **Alternatif akış:** Boş bir oda seçilirse kayıt silme düğmesi çıkmaz, işlem yapılamaz

7.3 Diyagramlar ve Hocanın Geri Bildirimi

Sınıf diyagramında veri tabanı ve farklı sınıflar arasındaki ilişkiler, metotlarla birlikte verildi. Ardışıl diyagramda bütün senaryolar tek bir diyagramda birleştirilmişti. Burada diyagram sayısı senaryo sayısının altındadır: $1 < 11$. Hoca bu sunumda diyagram sayısına ayrıca değinmedi, ama lastik oteli projesinde verdiği kurala göre her senaryonun en az bir ardışıl diyagram üretmesi gerekir.

Yapı diyagramında işlemler arasındaki veri geçişleri gösterildi. Örneğin otomasyon işlemi, otel işlemlerinden gelir ile yemek ve etkinlik maliyeti verisini alıp istatistiksel işlemlere veriyor, oradan gelen grafik verisiyle grafikleri gösteriyordu. Varlık-ilişki diyagramında dikdörtgenler varlık, eşkenar dörtgenler ilişkiydi.

Grubun raporu bitince hoca iki geri bildirim verdi:

> **Hocanın vurgusu:** Nesne yönelik yaptıysanız yapı diyagramına gerek yoktur.
>
> **Hocanın vurgusu:** Analiz modelinin ve tasarım modelinin sınıf diyagramlarını ayrı tutun. Önce yalnız kavram sınıflarından oluşan basit bir kavramsal diyagram çizilir. Sonra davranışlar eklenir ve gerçek yazılım sınıflarından oluşan tasarım diyagramı çizilir.

7.4 Sınanan Durumlar ve Gösterim

Müşteri değerlendirmesinde iki eksik belirlendi. Herhangi bir çalışan beş modülün hepsine erişebiliyordu, bir yönetici paneliyle erişim kısıtlanabilirdi. Ayrıca MySQL tarafında güvenlik iyileştirmeleri yapılabilirdi. Müşteri olmadığı için istatistik modülü, test aşamasında elle girilen verilerle pasta, çizgi ve sütun grafikleri üzerinde gösterildi.

Demoda oda kaydı ekranında 100 oda görülüyordu, yeşil boş anlamına geliyordu. Önce bir odaya ileri tarihli 5 günlük rezervasyon yapıldı. Yemek tercihi dört seçenekten biri olmak zorundaydı.

Sezona göre fiyat: Ocak ve Aralık aylarında fiyat en düşük düzeyde tutuluyordu. Taban fiyat 800 TL idi. Temmuz'un 5'i ile 10'u arasında fiyat yaklaşık 2,5 katına kadar çıkabiliyordu. Demoda tek kişi için önerilen fiyat 1871 TL olarak görüldü.

- Üst sınır (ara adım eklendi): $800 \times 2{,}5 = 2000$ TL
- Demodaki oran (ara adım eklendi): $1871/800 \approx 2{,}34$

Sonra aynı odaya 12 günlük kayıt denendi. Bu süre ileri rezervasyonla çakıştığı için sistem "bu odaya ilgili tarihte zaten bir rezervasyon mevcut" hatasını verdi. Demonun amacı, çakışma kontrolünün çalıştığını sınır durumla göstermekti.

Oda temizlik ekranında oda boş, dolu-temiz, dolu-kirli ya da arızalı olabiliyordu. Arızalı odada problemin nedeni belirtiliyordu.

Soru: Yani her gün o gün otelde kim varsa oda kaydından onun verisi çekiliyor mu?

Cevap: Evet. Her gün temizlik gerektiği için dolu odalar otomatik olarak kirli işaretleniyor.

Etkinlik takviminde etkinlik, tarih ve fiyat girilebiliyordu. Tarihi geçmiş bir etkinlik silinirse sistem onu yapılmış sayıp istatistiklere ekliyordu. Yemek takvimi ise bir veri setinden vejetaryen, vegan, çölyak ve normal menüler için gün içinde değişmeyen rastgele yemekler üretiyordu.

8. Araç Kiralama Uygulaması

8.1 Kapsam ve Fizibilite

Bir grup, bir araç kiralama şirketinin süreçlerini yönetmek için uygulama geliştirdi. Müşteri dört modül istedi: araç kayıt, kiralama, müşteri yönetimi ve yetkili yönetici modülü. Teknik fizibilitede Dart tabanlı Flutter seçildi. Başlıca neden, tek projeyle Android, iOS ve web üzerinde çalışabilmekti. Zaman fizibilitesinde waterfall modeli kullanıldı. Yasal fizibilitede KVKK maddelerinin yanında araçta sorun çıkması durumunda izlenecek adımlar da belirtildi: Sorunun kaynağına göre farklı adımlar izleniyordu.

8.2 Örnek Fizibilite Karşılaştırması

Üç sistem karşılaştırıldı:

Ölçüt	Sistem 1	Sistem 2	Sistem 3
Veri tabanı desteği	Öngörülen kullanıcıyı karşılamaz	Yeterli	Gereğinden fazla
Sorunlara önlem	Yok	Var	Var
Maliyet (TL)	229 643	264 500	281 500
Geri dönüş (TL)	350 000	600 000	600 000
Net fark (TL), ara adım eklendi	120 357	335 500	318 500
Süre	2 aydan az	2 ay	2 aydan fazla
Operasyonel	Rahat kullanıma uygun değil	Tüm ihtiyaçları karşılar	Sistem 2 ile aynı

Tablodan şu okunuyor: Birinci sistem en ucuz ve en kısa olanıdır, ama teknik ve operasyonel olarak yetersizdir. Üçüncü sistem gereğinden güçlü olup aynı geri dönüşü daha yüksek maliyetle sağlar. İkinci sistem yeterli kapasiteyi en yüksek net farkla verir.

8.3 Diyagramlar

Kullanım durumu diyagramında müşteri üye oluyor, giriş yapıyor, bakiye ekliyor, araç bilgilerini görüp kiralıyor ve ödeme yapıyordu. Bakiye yetersizse işlem olumsuz sonuçlanıp iptal ediliyordu. Yönetici kayıtlı üyeleri görüyor, araç bilgilerini güncelliyor ve kiralama taleplerini kaydediyordu.

Sınıf diyagramında `Car`, `User`, `Main` ve `Rent` sınıfları vardı. Grup, Dart'ta bazı metotların bir sınıf içine konmadığını, bu yüzden diyagramda tam gösterilemediğini belirtti. Sınıflardaki özellikler gizliydi ve erişim için set, get metotları vardı.

Ardışıl diyagram şu akışı izliyordu: Yeni kullanıcı eklenir, kullanıcı giriş yapar, show car ile araçlar listelenir, bakiye eklenir, kiralama isteğinde ödeme metodu çalışır, sonra arabanın uygunluk durumu dolu yapılır. Yapı diyagramında örneğin kullanıcı bilgisi girildiğinde sistem doğrulama döndürüyordu. Varlık-ilişki diyagramında aracın modeli, plakası, markası, yakıt türü ve müşterinin adı, soyadı, bakiyesi tutuluyordu.

8.4 Uygulamanın Gösterimi

Uygulamada normal kullanıcı ve yönetici olmak üzere iki kullanıcı tipi vardı. Yeni kullanıcının doğrulama talebi yöneticiye iletiliyordu. Normal kullanıcı araçları görüyor, cüzdanına para yatırıyor ve araç için kiralama süresi seçiyordu. Örnekte iki günlük kiralama sonrası bakiye düştü, aynı araç ikinci kez kiralanmak istendiğinde müsait olmadığı bildirildi. Yönetici kullanıcıları ve bakiyelerini görüyor, yeni araç ekleyebiliyordu. Sunucuya bağlanılamadığı için veriler yerelde tutuluyordu.

9. Spor Salonu Ağı

9.1 Amaç ve Kapsam

Başlıkta anılmasa da derste bir proje daha sunuldu. Bu proje, sporcu kaydından antrenman planına ve salon istatistiklerine kadar işlemleri tek sistemde toplamayı hedefliyordu. Asıl amaç, sporcu, salon sahibi ve antrenörün etkileştiği geniş bir ağ kurmaktı. Sporcu salon ya da antrenör seçerken hangi faktörlere bakacağını uygulamada görecekti.

Süreç toplantılarla ilerledi: 5 Mart'ta görev dağılımı ve yol haritası, 14 Mart'ta fizibilite ile backend başlangıcı, 14 Nisan'da backend tamamlandıktan sonra frontend teknolojisi kararı, 15 Mayıs'ta frontend sonrası uygulama üzerinde çalışma. Kapsam belgesinde ürün kalitesi, zaman, kaynak ve maliyet başlıkları vardı. Ekip üyelerinin asıl görevleri dışında gerektiğinde başka roller de üstlenebileceği planlandı.

9.2 Fizibilite ve Gelir Modeli

Üç sistem puanlandı, birinci sistem seçildi. Teknik fizibilitede Windows, MySQL, backend için Java Spring Boot, frontend için React kullanıldı. Geliştirme ortamları Eclipse, VS Code ve MySQL Workbench idi.

- **Sosyal:** Kullanıcı salona gitmeden bilgi alıp kayıt olabilir. Diğer kullanıcıların değerlendirmeleri sayesinde salon hakkında sosyal kanıta ulaşır.
- **Yasal:** Salon ile sporcu arasında anlaşma gerekir. Sporcu ve antrenörden alınan kişisel veriler KVKK'ya uygun saklanmalıdır.
- **Ekonomik:** Donanım ekibin kendi donanımı olduğu için gidere yansımıyor. Sunucu sayısı üye sayısıyla artacak.

Bu projenin ayırt edici yanı, gelirin doğrudan satıştan gelmemesidir. Plan şöyledir:

1. **İlk dönem:** Salonlardan uygulamada görünmek için aidat alınmaz. Hedef, en az kârla en çok kullanıcıya ulaşmaktır. Böylece piyasa uygulamaya kayar ve salonlar uygulamada bulunmak zorunda hisseder.
2. **Piyasaya hâkim olunca:** Salonlardan aidat ya da sporcu kaydında hizmet bedeli alınır.
3. **Sonraki dönem:** Öne çıkmak isteyen salonlarla sponsorluk yapılır, markalardan reklam alınır.

Tahmini gider analizi ve gelir-gider grafiği yapıldı. Plana göre başa baş noktası 12. aydır, proje bir yılını tamamladıktan sonra net kâra geçecektir.

9.3 Diyagramlar

Gantt şeması, kavramsal UML diyagramı ve üç ayrı veri akış diyagramı gösterildi. Antrenör akışında üye olma ve antrenman programı ekleme, sporcu akışında üye olma ve antrenör kiralama, salon akışında müşteri kaydı ekleme, kayıtlı müşterileri görüntüleme, yorum ve puanları görüntüleme vardı. Varlık-ilişki diyagramında sporcu, spor salonu ve antrenör varlıkları bulunuyordu. Sporcu ile antrenör salona kaydolabiliyor, salonla bağlantı kurabiliyordu.

> **Ders sonrası not:** Hoca bu sunumda diyagram seçimine ayrıca değinmedi. Önceki grupların geri bildirimleri büyük olasılıkla bu proje için de geçerlidir: Backend Java ile yazıldıysa kavramsal sınıf diyagramının yanına veri akış diyagramı koymak, yaklaşım tutarlılığı sorusunu doğurur.

9.4 Backend ve Frontend Bağlantısı

Hoca, backend ile frontend'in ayrı ayrı değil, birlikte çalışan sistem olarak gösterilmesini istedi. Ancak frontend, backend'den veri çekerken CORS hatası veriyordu ve iki gündür çözülememişti. Bu yüzden istekler Postman ile gösterildi: GET isteğiyle antrenör ve sporcu verileri çekildi, POST isteği gönderildi. Eclipse'te denetleyici sınıflarındaki GET, PUT ve silme eşlemeleri açıklandı.

> **Ek bilgi (derste anlatılmadı):** CORS, tarayıcının bir sayfanın farklı kaynaktaki (origin) bir sunucuya yaptığı isteği denetleme kuralıdır. Hatanın Postman'de görülmemesi büyük olasılıkla bu nedenledir, çünkü Postman tarayıcı değildir ve bu denetimi

uygulamaz. Spring Boot tarafında izin verilen kaynakların tanımlanması genellikle sorunu çözer.

10. Projeleri Yan Yana Görmek

Sunumların kullandığı araçlar ve sayısal özetler aşağıdadır:

Proje	Teknoloji	Yaklaşım	Fizibilite sonucu
Restoran	Python	Ağırlıkla yapısal: DFD, yapı, ER	Gider yaklaşık 35 bin dolar
Lastik oteli	Java, MySQL	Nesne yönelik	Gider 78 000 TL, fiyat 120 000 TL, matris 89 puan
Sinema	Python, Django, SQLite	Hocaya göre nesne yönelik değil: DFD, yapı, ER	Bütçe 4000 dolar
Fitness center	Java, NetBeans, MySQL	Nesne yönelik hedefli, sınıflar ekran bazlı	Süre 53 gün, matris 66 puan, geri ödeme 3 ay
Pet shop	Java, MySQL	Sınıf, ardışıl, ER	Gider 123 400, geri kazanım 2,5 ay
Otel	Java, MySQL	Nesne yönelik, ek olarak yapı diyagramı	2 aylık proje, 11 senaryo
Araç kiralama	Flutter, Dart	Sınıf, sequence, yapı, ER	Seçenekler 229 643, 264 500, 281 500 TL
Spor salonu ağı	Spring Boot, React, MySQL	Kavramsal UML, DFD, ER	Başa baş 12. ay

Hocanın geri bildirimlerinden çıkan denetim listesi şudur:

- Senaryo cümleleri etken ve özneli mi?
- Senaryonun birinci aktörü diyagramdaki aktörle aynı mı?
- Yaklaşım tek ve tutarlı mı?
  - Nesne yönelik: kullanım durumu, kavramsal sınıf, ardışıl, tasarım sınıf diyagramları
  - Yapısal: veri akış diyagramı, yapı diyagramı
- Ardışıl diyagram sayısı en az senaryo sayısı kadar mı?
- Sınıflar kavramlardan mı oluşuyor, ekranlardan değil mi?
- İş katmanı sunum katmanından ayrı mı?
- Analiz ve tasarım sınıf diyagramları ayrı mı?
- Veri tasarımı yalnız ürün adıyla bırakılmamış, gerçekten yapılmış mı?
- Hata mesajı kullanıcıya yol gösteriyor mu?

Hoca sunum tarzı için de küçük ayarlar yaptı: Senaryolar ve tablolar tek tek okunmamalı, önemli noktalar özetlenmeli. Terimler de doğru kullanılmalı, “yeni girişim” örneğinde olduğu gibi. Hoca, sunumda geçen “sallıyorum” gibi günlük ifadelere de itiraz etti.

Dersin Toparlaması

Bu derste, dönem boyunca görülen analiz ve tasarım araçlarının gerçek projelerde nasıl kullanıldığı görüldü. Fizibilite tarafında projeler ortak bir mantık izledi: Alternatifleri belirle, teknik, ekonomik, zaman, sosyal, yönetim ve yasal başlıklarda karşılaştır, en yüksek puanlı olanı seç, kişisel veri varsa KVKK’yı ele al. Fitness center matrisi ile araç kiralama karşılaştırması, bu seçimin neden tek bir ölçüte bırakılmadığını açıkça gösterdi.

Hocanın geri bildirimleri daha çok modellemenin tutarlılığına yöneldi. Nesne yönelik bir projede kavramlar sınıflara dönüşmeli, senaryolar ardışıl diyagramlara karşılık gelmeli, analiz ve tasarım modelleri ayrı tutulmalı ve iş kodu arayüzden ayrılmalıdır. Yapısal bir projede de

nesne yönelik diyagramları eksik kalmasın diye eklemek gerekmez. Bir sonraki derste kalan grupların proje sunumları devam edecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.